Literature Review: Concurrency Control for Seat Reservation in Web-Scale Booking Systems

Scope. This review supports the CineBook research project. It surveys how database-backed applications prevent the same item from being sold twice, what each approach guarantees, how each fails, and how such systems should be measured. It closes by identifying the gap the project's experiments address. Sources were selected from database systems research (SIGMOD, VLDB, ACM TODS), systems measurement methodology (ASPLOS, OOPSLA, SC), independent correctness testing (Jepsen), and documented industry incidents.

1. The problem: hot-spot contention

A ticket sale concentrates demand on very few records at the same moment. Two widely reported launches show the scale: Ticketmaster reported 3.5 billion system requests during the 2022 Eras Tour presale, about four times its previous peak [15], and BookMyShow's platform went down at the start of the 2024 Coldplay India sale, with around 13 million users reported for roughly 180 000 tickets [16]. Most production systems put a virtual queue in front of the database to limit how many buyers reach it at once, but a queue only reduces contention; it cannot remove it. Whatever passes the queue still competes for the same seat rows, and correctness depends on how those competing transactions are controlled.

2. Why naive code fails: isolation anomalies

The standard SQL isolation levels are defined through the anomalies they rule out. Berenson *et al.* [1] showed that the original ANSI definitions are ambiguous and incomplete, and introduced clearer anomaly definitions, including the *lost update*: two transactions both read a value and both write a new value based on it, so one write silently overwrites the other. Double booking is a lost update. If two buyers read "seat available" and both write "seat held", both believe they own the seat.

Most databases, PostgreSQL included, default to READ COMMITTED, which permits this anomaly for read-then-write logic. The same paper also defined snapshot isolation, which removes lost updates for transactions writing the same row but still allows *write skew*, where transactions read overlapping data and write different rows.

3. Approaches and their trade-offs

Table 1 summarises the approaches discussed in this section.

TABLE 1: Approaches to preventing double booking

Approach	Guarantee	Typical failure mode	Main cost	Key sources
Application check then write ("feral")	none under concurrency	lost update; oversell	none until it fails	[2], [4]
Conditional write (UPDATE … WHERE free)	one winner per row	multi-row all-or-nothing needs care	waits on row lock	[6]
Pessimistic locking (SELECT … FOR UPDATE)	serialised access to locked rows	deadlock if lock order differs	lock waits, extra round trip	[1], [6]
Optimistic validation (version check)	aborts on conflict	wasted work under high contention	retries	[5]
Serializable isolation (SSI)	serializable executions	aborts, including false positives	retries, tracking overhead	[3], [10]
Coarse / external locks (advisory, Redis)	mutual exclusion if correctly used	serialises unrelated work; timing assumptions	throughput	[11]

Application-level checks. Bailis *et al.* [2] analysed 67 open-source Ruby on Rails applications and found that integrity is enforced mainly through framework validations and similar application-level ("feral") mechanisms, which were used more than 37 times as often as transactions. Many of these checks are unsafe when requests run concurrently. Warszawski and Bailis [4] later showed the consequence is not only theoretical: by triggering concurrent requests on purpose, they found 22 exploitable attacks in 12 real applications, and noted that low everyday traffic hides these defects.

Conditional writes. A single statement that both tests and changes a row moves the decision into the database. In PostgreSQL's READ COMMITTED mode, an UPDATE that meets a row locked by a concurrent writer waits for that transaction to finish. If it committed, the WHERE condition is evaluated again against the new row version, and the row is skipped if it no longer qualifies [6]. A guarded update such as "set held where status is available" therefore lets exactly one competitor succeed per row. Requests for several seats need an extra all-or-nothing check, for example comparing the number of updated rows with the number requested.

Pessimistic locking. Locking rows before reading them serialises competing transactions. The classical risk is deadlock: if one transaction locks A then B and another locks B then A, each waits for the other. Databases detect and break deadlocks by aborting a victim, and the usual prevention is to always lock rows in one agreed order.

Optimistic concurrency control. Kung and Robinson [5] proposed executing transactions without locks and validating them before writing: if data read by the transaction changed in the meantime, the transaction is aborted and retried. Because no locks are held, there is no deadlock, but under heavy contention many transactions do work that is then discarded.

Serializable snapshot isolation. PostgreSQL implements its SERIALIZABLE level with SSI [3], which tracks read-write dependencies between concurrent transactions and aborts one when a dangerous pattern appears. Its authors report performance close to plain snapshot isolation on read-heavy workloads. SSI tracks conflicts at a coarser granularity than individual values, so it can abort transactions that did not truly conflict, and applications must be prepared to retry. Independent testing by Jepsen found G2-item anomalies in the serializable mode of PostgreSQL 12.3 under normal operation [10], a reminder that isolation guarantees deserve empirical checks rather than assumption.

Coarse and external locks. A single lock per show is simple and correct, but it serialises every booking for that show, including bookings for different seats. Locks held outside the database, such as Redis-based distributed locks, add timing assumptions. Kleppmann argued that a lock used for correctness needs fencing tokens checked by the storage system, because a paused process can keep acting after its lock has expired [11]; the Redis authors dispute parts of that critique.

4. How such systems should be measured

Performance results for concurrent systems are noisy and easy to misread. Mytkowicz *et al.* [9] showed that seemingly irrelevant details of an experimental setup can bias measurements enough to reverse a conclusion, and proposed randomising the setup as a defence. Georges *et al.* [7] argued for repeated runs and confidence intervals instead of single best or average results. Hoefler and Belli [8] reviewed 120 papers, found reporting practice weak, and recommended rules that include stating the environment fully, reporting variability, and using non-parametric statistics when data are not normally distributed. Dean and Barroso [12] explain why tail latency, rather than the average, determines user experience in large services. Serverless platforms add their own variability: Wang *et al.* [13] measured more than 50 000 function instances on major platforms and documented cold-start delays and weak performance isolation.

For correctness outcomes that are counted (for example, the share of trials with a double booking), Brown, Cai and DasGupta [14] showed that the common normal-approximation interval behaves poorly and recommended the Wilson interval. When no failures are observed in n trials, the "rule of three" gives an approximate 95% upper bound of $3/n$ on the failure rate.

5. Research gap and how the project addresses it

The literature explains well *why* read-then-write code fails and *what* each mechanism guarantees in principle. Three questions are rarely answered with reproducible measurements:

1. **Pattern-specific comparison.** How do these strategies compare for the seat-reservation pattern specifically, which combines one hot row with all-or-nothing multi-row holds?
2. **Modern deployment stack.** How do they compare on a serverless application with a managed database, where network round trips rather than CPU dominate the cost of each transaction?
3. **Test platform.** Does the platform used for testing change whether the defect can be observed at all? Many projects test against embedded or single-connection databases.

The CineBook study addresses these questions with a pre-registered set of experiments: seven strategies, repeated trials, randomised execution order, non-parametric statistics with confidence intervals, and end-to-end tests against the deployed system. The design and results are in `docs/methodology.md` and `docs/research-report.pdf`.

References

[1] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, "A critique of ANSI SQL isolation levels," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 1995, pp. 1–10.

[2] P. Bailis, A. Fekete, M. J. Franklin, A. Ghodsi, J. M. Hellerstein, and I. Stoica, "Feral concurrency control: An empirical investigation of modern application integrity," in *Proc. ACM SIGMOD*, 2015, doi: 10.1145/2723372.2737784.

[3] D. R. K. Ports and K. Grittner, "Serializable snapshot isolation in PostgreSQL," *Proc. VLDB Endow.*, vol. 5, no. 12, pp. 1850–1861, 2012, doi: 10.14778/2367502.2367523.

[4] T. Warszawski and P. Bailis, "ACIDRain: Concurrency-related attacks on database-backed web applications," in *Proc. ACM SIGMOD*, 2017, doi: 10.1145/3035918.3064037.

[5] H. T. Kung and J. T. Robinson, "On optimistic methods for concurrency control," *ACM Trans. Database Syst.*, vol. 6, no. 2, pp. 213–226, 1981.

[6] PostgreSQL Global Development Group, "Transaction isolation," *PostgreSQL Documentation*, ch. 13.2. Available: https://www.postgresql.org/docs/current/transaction-iso.html

[7] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. ACM OOPSLA*, 2007, pp. 57–76.

[8] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. SC*, 2015, doi: 10.1145/2807591.2807644.

[9] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!," in *Proc. ASPLOS*, 2009, pp. 265–276.

[10] K. Kingsbury, "Jepsen: PostgreSQL 12.3," 2020. Available: https://jepsen.io/analyses/postgresql-12.3

[11] M. Kleppmann, "How to do distributed locking," Feb. 8, 2016. Available: https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html

[12] J. Dean and L. A. Barroso, "The tail at scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[13] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking behind the curtains of serverless platforms," in *Proc. USENIX ATC*, 2018, pp. 133–146.

[14] L. D. Brown, T. T. Cai, and A. DasGupta, "Interval estimation for a binomial proportion," *Statist. Sci.*, vol. 16, no. 2, pp. 101–133, 2001.

[15] "Ticketmaster cancels public sale of Taylor Swift tour tickets," WSB-TV / Cox Media Group, Nov. 2022.

[16] "Coldplay India tour 2025: Band adds 3rd Mumbai show amid ticket chaos on BookMyShow," *Business Today*, Sep. 22, 2024.